Complete description of the project

Here is the current stable version of the nightsquirrel question/answer Flask project.

Note: the site uses pyBabel to manage 2 languages: Italian and English. Italian is the default language: the user can switch on the fly to decide which language to see on the pages.

Users and roles

The tbl_u_user entity models the general user. The 4 main actors are: student, tutor, admin, and payer.

Tutor/admin separation exists.

Furthermore, there is a "payer" role, usually one of the student's parents, which is assigned to a user with the usr_is_payer flag set to true (as a side note, a student can auto-pay herself if she owns a PayPal account.)

Main use case

1. The (registered) user creates a question, filling in the metadata and the question body via Quill (as for a recent implementation, she is able to upload a picture or pdf of the problem).
    - **1.a.** If the uploaded element is an image, then the user is able to crop/rotate the image (implemented via Cropper.js)
    - **1.b.** If the element attached to the question is an image, the AI can try and extract the content of the image; the content is placed in the empty Quill editor, so that the user can enhance/edit it and send this parsed content as main part of the question.
2. The question is linked to a ticket, which is created contemporarily, to ease the management of the question itself.
3. The question/ticket is analysed by a module that is able to compute a price for the question (it may be supervised by the admin).
    - **3.a.** If the question data (text, description, title etc.) is incomplete, or inconsistent, and needs clarification, the platform notifies immediately to the student that the question must be revised and clarified. This feature involves an AI engine to check question sustainability!
4. If the question is reasonable, the user receives a notification and sees the pricing policy for the question and decides to accept it or not. For the moment, let's suppose that if she does not accept, she is left with a goodbye. SEE BELOW FOR THE "EXPANSION" OF THIS USE CASE POINT
    - **4.a.** If the student has not set up a payment method via the payer, she is asked to do that. For the moment, all payments are managed via PayPal. See the Payer use case at the bottom of this description, but note that she may be self-paying
5. If she accepts the quote, the tutor or the admin can manage the question/ticket.

    - **5a.** Stopping/deleting a question/ticket. The student can, up to this point, "hard" delete a question to stop the ticket pipeline. For the delivered state onwards, the user can "soft" delete or hide the question from her profile and list. In this latter case, the question and ticket related entities remain in the database.

6. When the question/ticket is in the delivered state, the user receives a notification. CRUCIAL: at this point, the payment method associated with the user profile is used to charge the fee to the credit card via the Payer's PayPal account.

7. The user sees the answer and may: close the ticket/question or "chat" with the tutor via comment exchange.

    - (This latter feature requires a minimal engine, which is implemented using Quill for editing+maths capabilities. Furthermore, a very nice real-time interaction is achieved by integrating Pusher Channels!).

The pricing mechanism is synchronous for now, in pricing.py. Anyway it is powerful enough to produce a json object with multi-dimensional values, aka axis.

States, transitions and notification

The possible states for this use case are defined in **states.py**.

Note that even if the entities with a "state" column are 3, i.e., tbl_q_question, tbl_t_ticket and tbl_q_answer, to keep things as simple as possible, only the ticket state is used as the source of truth.

For many transitions of the ticket states, there are corresponding notifications via email - notifications enabled by an external mailer service. See notifications.py. Note that every time the student accepts a quote for a ticket, the payer should be notified for the very imminent payment (which happens when the ticket is delivered).

User subscription and profile management

Point 1 of the main use case is supposing that the user/student is registered and has a subscription. The subscription process is included in the project, with usual login/subscribe/forgot password operations. When the user is subscribed and logged in, she has access to a profile page; this page lets her:

- change username
- change main email (she will receive a new login, via a OTP mechanism!)
- change password
- attach PayPal payment method
- delete subscription.

Admin use case

The use case for the admin is as follows:

1. The admin can manage users' features (and the "validity" of every single user)
2. The admin can see all questions and assign a question to a tutor
3. For certain special cases, the admin can manually quote a question

4. The admin manages the Library/Book Wall, with all related entities
5. The admin manages the example page (localized ita + eng)
6. The admin manages the "Ideas" and related entities (see related section)

What happens when the Admin sets the usr_isvalid = false? It depends on the user "type":

- If is_student:
  - cannot create questions
  - cannot accept quotes
- If is_payer:
  - cannot be used for charging
- If is_tutor:
  - cannot claim/in_progress/deliver
- If is_admin: nothing happens

Payer use case

Here is the payer use case.

First: if the payer is the student herself, she has to connect a PayPal account (and get a vault id behind the scenes), so that she can accept the offer for the question (point 4 of the user's main use case).

But the main alternative is this: the student "invites" a Payer (a parent, a guardian etc): the payer is automatically subscribed to the platform, via a dedicated page

Then the Payer must:

- log in to the site
- connect his PayPal account, activating a vault id behind the scenes.

From that point on, the student "connected" to the payer can accept the fees for the questions.

Attached documents and embedded images

The document entity, containing the (optionally) uploaded images/pdfs for the questions, is managed via a dedicated table and an S3 bucket.

Pricing generation for the question

Quote generation happens on question “Save” (not per keystroke) only if tkt_state is in pre-acceptance states (e.g. NEW, QUOTED, NEEDS_REVIEW). After acceptance, quote payload is immutable.

On Save, the backend builds a normalized economic input object and computes tkt_quote_signature (new field).

If the signature is unchanged, the system keeps the existing tkt_quote_payload and does not bump tkt_quote_version or tkt_quoted_at. If the signature changed (economic inputs changed), backend regenerates tkt_quote_payload (new field: JSONB snapshot containing axes definitions like speed, depth and a curated list of discrete options with stable options[*].id), updates tkt_quoted_at, and bumps tkt_quote_version.

tkt_quote_points stays as a derived scalar summary for quick filtering.

The UI renders sliders for each axis using the axes data inside tkt_quote_payload and maps the slider position to one of the discrete options[*] (preview).

When the student confirms, the chosen options[*].id is stored in tkt_selected_option_id and its price_cents is stored in tkt_selected_quote_cents.

On Accept quote, the ticket transitions to accepted state, sets tkt_accepted_at, and creates the payment record using tkt_selected_quote_cents (not recomputed). Any later payment uses the frozen selected amount + option id for consistency and auditability.

References

For the moment, references can be of 4 types: books, articles (papers), video links and web links.

References can be created and/or attached to the question by the student user. They can also be created and/or attached to an answer by the tutor user.

The admin user can create a possibly big number of references so that other users can browse them and attach them or use them. Thus the admin manages a sort of big library of references. This allows the creation of a section of the site which is called "Leo's book wall", where the student can see many wonderful books to be inspired -- these books have been read and studied by Leo, so that he can connect and suggest inspiration to the students.

Each reference has a thumbnail and a bigger cover image.

The admin should be able to see on the dashboard all references. The user should be able to manage her references, but see and attach also the admin's library reference.

At the moment there is a way (for the admin only!) of getting metadata and images about new books via scanning/inserting ISBN numbers. When the admin uses this feature, it can scan the ISBN from the book: an initial draft of the book with some info ends up in a draft books table.

Then the admin has the option to let an AI complete the book information for each "draft book", adding Person and Publisher entities in the correct way (if found and available). When done, the admin can consolidate the new book reference as if it was inserted by hand.

As a final feature, referemces can be flagged with one or more of the following states:

- is_current: Leo/the admin is currently reading the reference
- is_recent: Leo/the admin has recently read the reference
- is_crucial: it highlights references of fundamental importance.

The person page (and list)

The person entity is connected to references: the obvious example is the author that has written one or more books of the library. When a person name is present in one of the pages of the site, the name links to a dedicated **person page** that showcases all the references of that person.

THere is also a "People" page, a listing of all items of the "persons" table. The list features each person, with a summary of related references (the list is interactive)

Tags

The tag entity is implemented; tags can "decorate" semantically questions and references. Students, tutors and admin can attach one or more tags to a question: the student to her questions, the tutor to assigned questions, the admin to every question.

The student and the tutor can add tags to their references. The admin can add tags to any reference.

Tags are used (read only 😃 in the library to help clarify the content of a book / reference.

For the moment, only the admin can create a tag.

The library show a tag-cloud with the list of all tags; each tag is linked and the link leads to a **tag page**, where there is a list of all references to which the tag is connected.

Examples and example page

The platform has a page showcasing examples. These examples, as content of the examples page, are managed by the admin, via a specialized section of the backoffice.

The examples are grouped by "question types", and are localized (ita, eng). Fruthermore, they are diveded by difficulty/grade (so that the user can choose which level to see)

AI features

All AI capabilities are powered by the **Anthropic API** (Claude models), implemented in `ai_provider.py` and `question_analysis.py`. The system is designed to fail-open: if the API is unavailable or returns an unexpected response, the platform continues to work normally without AI assistance.

1. Image OCR and content extraction

When a student uploads an image as the main body of a question, Claude vision (model: `claude-sonnet-4-6`) analyses the image and extracts its full content into structured segments: plain text and mathematical formulas expressed as LaTeX (KaTeX-compatible). The result is converted into a Quill delta and placed directly into the editor, so the student can review, correct, and enrich it before submitting. This is particularly useful for handwritten exercises or scanned textbook pages containing equations.

The extraction is confidence-scored (0–1): crisp printed text scores near 1.0, messy handwriting near 0.4. The confidence value is stored alongside the delta for future reference.

2. Question sustainability gate

Every time a student saves a question (in pre-acceptance states), an AI analysis is run in the background (`analyze_and_gate_question` in `question_analysis.py`). Claude evaluates whether the question is clear, complete, and answerable by a tutor. The gate distinguishes two paths:

- **Vision-aware path:** if the question body contains inline images (embedded directly in the Quill editor), the text blocks and image URLs are passed together to Claude as a multi-modal message (`claude-sonnet-4-6`). This ensures that image-heavy questions — where the text field may be nearly empty — are still evaluated correctly.

- **Text-only path:** for questions without inline images, the Quill delta is converted to a plain-text representation (with LaTeX formulas preserved in brackets) and passed to a text-only call (`claude-sonnet-4-5`).

The AI returns two sections:

- **`input_suitability`:** a boolean `is_quotable` flag and, if the question is not suitable, a short friendly message in Italian (`student_hint`) explaining what needs to be fixed. Possible issue codes: `incomplete_statement`, `blurry_image`, `nonsensical_text`, `too_vague`, `off_topic`, `multiple_unrelated_questions`.
- **`semantic_analysis`:** predicted subject, school type, grade, pedagogical difficulty, and resolution complexity. These predictions are stored on the question record and can help the admin and tutors prioritize and route questions.

The full AI response is also stored in the ticket for audit and future use.

3. Person biography enrichment

The admin can enrich person records with short bilingual biographies (Italian and English) via an AI-assisted workflow. From the admin persons list, an **Enrich** button appears next to any person with at least one missing caption. Clicking it triggers a two-step process:

1. **Wikipedia lookup:** the system queries both the Italian and English Wikipedia REST APIs for the person's name and retrieves the introductory extract from each.
2. **Claude formatting:** the Wikipedia extracts (or the model's own knowledge if Wikipedia finds nothing) are passed to Claude (`claude-sonnet-4-6`), which produces a short standardised caption in both languages. The format is: *(place of birth, date of birth – place of death, date of death)* followed by a one-sentence description. Tense is adapted automatically — present for living persons, past for deceased ones.

The generated captions are saved immediately and the admin is redirected to the person edit page to review and correct them before they go live on the public person page.

If the enrichment process fails, the caption of both languages is set to 'n.a.' so that one can recognize the attempt on a person, from the case where no attempt has ever been made.

4. Fuzzy person and publisher matching in ISBN import

When the admin processes a book draft created via ISBN scan, the system must map author names and publisher names from the external API to existing `tbl_r_person` and `tbl_r_publisher` records in the database. This matching follows a three-step cascade:

1. **Exact match:** case-insensitive lookup by first name and family name (or publisher name).
2. **AI-assisted fuzzy match:** if no exact match is found, a set of candidate records is retrieved from the database via a keyword/family-name search, and Claude (`claude-haiku-4-5`) is asked to identify whether any candidate is the same person or publisher as the name from the API. The model returns either the matching database ID or a "no match" signal.
3. **Create new:** if neither exact nor AI match succeeds, a new person or publisher record is created automatically.

This approach avoids duplicate entries while keeping the import pipeline fully automated.

5. Nobel attribuition via person "enrichment" and bulk

Each person in the library can be enriched with their Nobel Prize data. The system queries the Nobel Prize API v2.1 by family name, then applies a token-subset first-name matching algorithm to avoid false positives (e.g. a common surname shared by both laureates and non-laureates). When a match is confirmed, the prize year and category are stored in the `per_won_nobel` field (e.g. `"1992 – Economics"`; multiple prizes are semicolon-separated).

Enrichment is triggered from the admin panel on a per-person basis via a dedicated button ("Enrich" / "Re-enrich"), which also attempts to generate AI-written biographical captions if they are missing — but never overwrites captions that already exist. Nobel data is always refreshed from the API regardless.

For bulk operations (e.g. initial population or corrections after a matching fix), a standalone script `scripts/enrich_persons_nobel.py` can be run as a Heroku one-off dyno. It supports `--dry-run` to preview changes and `--force` to reprocess persons who already have Nobel data.

On the user-facing side, confirmed laureates are marked with a dedicated icon wherever they appear: on the person detail page, in the library's people accordion, and on every book reference card that lists them as primary author.

The "idea" concept

The "idea" entity captures the concept of a piece of knowledge, that can be a "postit" note, up to a complete article (or better, a whiteboard).

Each "idea" can be linked to one or more anwers; e.g. the pythagorean theorem - described by an "idea" - con be linked to any answer that may require the referenced theorem.

Furthermore, an idea can be loaded with one or more tags, and can be linked to one or more references.

The tutor and the admin can create and manage ideas. The idea creator, as user, is the owner of the idea and can modify it. The admin can edit any idea in the database.

Each idea has a connceted whiteboard. The whiteboard can be "reduced" to a single complex-text element (managed by a Quill minieditor). The whiteboard is a complex object that may contain one or more widgets: text-boxes, references representation, one or more hmtl texts (managed by Quill), inages, links etc. These widgets/ graphical elements may be connected by arrows.

Some technical details

The project is built with Flask, Python, JavaScript and Jinja2.

It integrates a number of components:

- Anthropic API for AI related features
- PostgreSQL (via psycopg2) as the relational database for the data model and knowledge-base
- PayPal API for online recurring payments
- Quill editor + KaTex as a mini-editor for questions and answers, with maths display and embedded image capabilities

- Pusher for the interactive comments section
- AWS S3 buckets and boto3 APIs to save/retrive persistent images
- Google Books API to get automatic info about book references
    - as a fallback, OpenLibrary APIs are used to retrieve info about books
- pyBabel for multilingual and localization support (ITA / ENG)
- cropper.js for client-side image crop/rotate and the preview of images to be uploaded
- Pillow to resample images for thumbnails
- Bootstrap for responsive layouts and basic styling
- Whikipedia API to get a short bio for the "Person" library entity
- Noble prize API are used to get the people that won the price
- an external custom mailer (based on MailerToGo) for email notifications
- other Python libraries (e.g. FlaskLogin, PyJWT etc.) to tackle all the needed technical and security details

The database definition for the database supporting the project can be found in

- /database scripts/1-create-database.sql (if needed)
- /database scripts/2-fill-database-nightsquirrel.sql
- /database scripts/3-payment-tables.sql
- /database scripts/4-document-table.sql
- /database scripts/5-comment-table.sql
- /database scripts/6-reference-tables.sql
- /database scripts/7-tag-tables.sql
- /database scripts/8-book-draft-tables.sql
- /database scripts/9-library-seed.sql (see related section, this file is generated)
- /database scripts/10-example-tables.sql
- /database scripts/11-example-seed.sql (see related section, this file is generated)
- /database scripts/12-idea-tables.sql

Other important information can be found in the /docs folder.

The site is deployed on Heroku, and all libraries that need installation are found in the requirements.txt file.

IMPORTANT: locally, the project uses a venv (named venv) for local installation of python libraries.

The `main.py` and main-private.py contain all the needed env vars, including PayPal and AWS credentials.

Admin special functions and their usage

The admin's dashboard has 3 special functions (with their respective buttons) that deal with the datbase rebase.

- the "Rebase SQL" button appends all *.sql files in the /database scripts folder to create/download a single file (nightsquirrel_seed.sql) with all the instructions needed to create a database from scratch;
- the "Library seed" button creates/downloads the 9th*.sql file which contains all references in the Library, and the relevant connected entities (tags included!);
- the "Example seed" button creates/downloads the 11th*.sql file which contains all examples;
- the "Purge S3" button removes "orhpans" refernce images from the nightsquirrel-reference-image AWS S3 bucket.

To rebase the database one should:

1. Create a new and complete version of the 9-*.sql file, using the "Library seed" with all reference data to be recreated in the database.
2. Create a new and complete version of the 11-*.sql file using the "Example seed" button, and replace the local copy before committing.
3. IMPORTANT: if the database is the online version, the "Library seed" and "Example seed" are called "on-line". This means the new sql files are downloaded locally. We need to replace the LOCAL and stale copies, and then commit/push/publish the new version of the site online; in this way the online versions will be in par.
4. Use the "Rebase SQL" to get the complete set of instructions to re-seed the database.
5. Use CLI or PgAdmin (or similar) and run the instructions of the complete .sql file.
6. Use "Purge S3" to remove any dangling images in the S3 bucket

Babel pipeline

The localization of all words, captions and phrases directly contained in the html templates are managed via pyBabel. Each time one of these strings is added or modified, the following pipeline must be executed:

1. Extract all translatable strings into the .pot catalog
    - `venv/bin/pybabel extract -F babel.cfg -k _l -o messages.pot .`
2. Update existing .po files with new/changed/removed strings
    - `venv/bin/pybabel update -i messages.pot -d nightsquirrel/translations`
3. Translate. Open nightsquirrel/translations/it/LC_MESSAGES/messages.po and fill in any new empty msgstr entries, remove any #, fuzzy markers
4. Compile: .po → .mo (the binary file Flask-Babel actually reads)
    - `venv/bin/pybabel compile -d nightsquirrel/translations`

Important: always run these commands from the project root with `.` as the input directory for the extract step. Using `nightsquirrel/` instead of `.` will produce an empty extraction and destroy the catalog.